Provas do fluxo

Por que cada peça importante do fluxo é do jeito que é: o que a pesquisa diz e o que foi medido em testes próprios. Atualizado em 22/10/2026.

Como ler. Cada seção traz a decisão, as pesquisas que pesaram (com link), os testes feitos aqui (com número) e os cuidados. "Erro escondido" é um erro colocado de propósito para ver se a etapa acha.

Cuidado geral. Os testes próprios usaram poucos projetos (dois reais, um em Python e um web em TypeScript), e a maior parte dos erros escondidos foi escrita por IA. Os números mostram direção, não uma porcentagem para cravar. O que só foi usado, sem medição, está no fim.

---

1. Dois modelos: um escreve, outro confere

Decisão. O Claude escreve specs e testes e confere o código. O Gemini (pelo `agy`) escreve o código e confere as specs.

Pesquisa. Um modelo tende a aprovar o próprio trabalho:

O que foi visto	Fonte
Claude 3.5 Sonnet e GPT-4o dão nota maior aos próprios textos e aos da mesma família	Play Favorites (2025)
Avaliando por lista de critérios, mais de 50% mais chance de aprovar um critério que falhou quando o texto é do próprio modelo	Self-Preference Bias in Rubric-Based Evaluation (2026)
Se o modelo não sabe quem escreveu, o viés quase some (1 tarefa de escrita)	Self- and Other-Labels Induce Bidirectional Bias (2026)

Outras lidas: Self-Preference Bias in LLM-as-a-Judge, Quantifying and Mitigating Self-Preference Bias.

Cuidado. Nenhuma fonte testou conferência de spec ou de código entre famílias. Não há medição de que o Gemini programa melhor que o Claude: o ganho vem de separar papéis.

---

2. O CLAUDE.md global

Decisão. Arquivo curto, só com o que o modelo não faria sozinho.

Pesquisa.

- Arquivo de contexto ajuda pouco e custa cerca de 20% a mais (Evaluating AGENTS.md, 2026).
- Modelos seguem bem até umas 100 a 250 instruções; depois caem (How Many Instructions Can LLMs Follow at Once?).
- Regra de estilo é a que mais falha (CodeIF-Bench, CIFE).

- Versões travadas no `uv.lock` em vez de teto de versão: versão menor também quebra (Breaking Changes in Software Ecosystems).
- O mesmo banco no desenvolvimento e na produção (12-factor).
- Type hints evitam 15% dos bugs, desde que um checador rode (Khan et al., TSE 2021).

Teste próprio. O mesmo modelo fez 3 tarefas com e sem o CLAUDE.md global:

Regra	Com o arquivo	Sem	Decisão
Gerar testes sem pedir	5/5	0/6	fica
Type hints em função pública	4/5	0/6	fica
Sem `print`	5/5	3/6	fica
Sem `else` depois de `return`	5/5	6/6	saiu (o modelo já faz)
Sem `except Exception: pass`	5/5	6/6	saiu (o modelo já faz)

---

3. Formato da spec

Decisão. Cada spec tem "Regras" no formato QUANDO... DEVE..., "Interfaces públicas" (o que outras specs usam, escrito uma vez só) e "Usa de outras specs" (só o nome).

Pesquisa.

- Spec inchada piora o agente em 27 pontos (Constraint Decay, 2026); spec estruturada melhora 13% (WiseSpec, 2026).
- Testes guiando a geração: acerto de 48% para 71% (TiCoder).

Teste próprio. 4 specs de um projeto real com 12 erros escondidos entre elas. Formato antigo: 30% dos erros pegos. Formato novo: 11 de 12 (92%), 0 alarme falso em 11 specs limpas.

Cuidado. 1 projeto; erros escritos pelo Claude.

---

4. Conferir as specs antes de implementar (`/spec-review`)

Decisão. Três camadas: um script acha o que comparar, o Gemini confere cada spec, e o Claude julga pares de regras que podem se contradizer.

Testes próprios.

Camada	Teste	Resultado
Script `conferir_specs.py`	16 erros escondidos (4 de cada tipo)	16/16, 0 alarme em spec limpa
Verificador: Claude	12 erros + 3 specs limpas	12/12, cerca de 59 mil tokens por spec
Verificador: Gemini	os mesmos, 2 rodadas	22/24, 0 alarme; fora da cota do Claude
Pares de regras (Claude)	41 contradições + 47 pares sem conflito	41/41 e 0/47 alarmes; em pares novos, 22/22

Decisão tomada com os números. Usar o Gemini custa cerca de 1 erro em 12 e economiza os 59 mil tokens por spec. O Claude fica de reserva.

Pesquisa. IA julgando conflito par a par acerta bem: Claude 3.5 Sonnet com 86,6% em instruções conflitantes (ConInstruct, AAAI). A fraqueza conhecida: números e sinais como < e > geram alarme falso.

Cuidado. Nos pares de regras, a pergunta foi ajustada depois de ver os primeiros casos.

---

5. Testes antes do código, escritos por outro ator

Decisão. O agente `escritor-testes` lê só a spec e escreve os testes antes de existir código. Quem implementa não pode mexer em `tests/`.

Pesquisa.

- Teste "certo por coincidência" (passa no código errado) é 7,2 vezes mais comum que teste falhando (Coincidental correctness).
- Ver os testes ajuda quem implementa: +12,8, +9,2 e +3,1 pontos em três testes (Test-Driven Development for Code Generation).
- Deixar os testes visíveis, mas só para leitura, reduz a trapaça (ImpossibleBench).

Teste próprio. 7 versões erradas de um código, escritas de propósito. Os testes do texto antigo do agente pegavam 6/7 (duas rodadas, o mesmo erro vivo). Com o passo "escolher valores que separam o certo do errado", 7/7. Numa sessão nova, com o agente real: 5/5, e 91/91 testes passando no código real.

---

6. O Gemini implementa, com os testes travados

Decisão. O Gemini escreve o código sem permissão para escrever em `tests/`. No fim, uma impressão digital (hash) dos testes é conferida.

Uso real. 8 specs. Num projeto web, 17 rodadas: os testes foram de 2.278 para 3.058 e o hash bateu em todas, ou seja, o Gemini nunca mexeu em teste.

Problema visto. Em 1 das 17 rodadas o Gemini fez um atalho de propósito (código só para o nome aparecer no arquivo). Os testes não pegaram; a leitura do Claude pegou.

---

7. O Claude confere o código

Decisão. Depois dos testes, o Claude lê o código contra a spec (`/implementar`) e o agente `revisor-codigo` lê o diff de novo no `/spec-close`.

Rede de testes. Num projeto com 760 testes, os testes sozinhos pegaram 56 de 65 erros escondidos (86%). Os outros 9 só a leitura pode pegar.

Teste próprio (21/10/2026). 9 códigos com um erro escondido que passa em todos os testes, mais 8 códigos certos. O Claude leu sem saber qual era qual. Meta fixada antes: pelo menos 85% e no máximo 1 alarme falso.

Quem lê	Erros achados	Alarme falso	Custo por leitura
Conferência do `/implementar`	8/9 (89%)	0	cerca de 95 mil tokens
`revisor-codigo`	8/9 (89%)	1 leve	cerca de 65 mil tokens

- Os dois perderam o mesmo erro: uma linha de checagem comentada num arquivo novo.
- Nos códigos "certos", acharam 5 bugs reais que ninguém tinha visto.
- Mesmos casos, em 10/10: Gemini 6/9; Opus pelo `agy` 4/6.

Cuidado. Um dos 9 erros deixa um parêntese sobrando e talvez nem passasse nos testes; sem ele, 7/8 (87,5%).

---

8. Testar os testes (mutantes)

Decisão. Depois da implementação, o Gemini propõe 5 erros pequenos no código; um script aplica cada um, roda os testes e desfaz. Erro que sobrevive vira aviso no relatório.

Pesquisa. O Google roda essa técnica em toda mudança, durante o code review, só nas linhas mudadas (*State of Mutation Testing at Google*, 2018, 6.000 engenheiros). A Meta faz o mesmo por mudança e 73% dos testes gerados foram aceitos (Meta ACH, 2025).

Teste próprio. Numa cópia de projeto real, 5 erros propostos em cerca de 3 minutos, 5/5 pegos pelos testes e os arquivos voltaram ao original.

Cuidado. Ainda não apareceu um erro sobrevivente num caso real.

9. O que saiu do fluxo, e por quê

O que saiu	Por quê
Um verificador externo especializado, na conferência do código	Meta de 90% (mínimo 85%). Teto medido: 82%, em 23 variantes de pergunta e 65 erros escondidos. Nos erros que os testes deixam passar, ele não achou nada que a leitura do Claude já não achasse.
Segunda leitura do código pelo Gemini	Pegou 6/9, com 1 alarme falso. Somar dois modelos não subiu o acerto num teste com 5 modelos e 150 casos (arXiv 2606.15689); nove juízes valem cerca de 2 votos, porque erram juntos (*Nine Judges, Two Effective Votes*, 2026).
Um segundo modelo do `agy` como reserva de cota	Tem a mesma cota pequena. Quando a cota do Gemini acaba, o fluxo para e pergunta.
Duas regras de estilo do CLAUDE.md	O modelo já seguia sem a regra (seção 2).

10. O que é usado, mas não foi medido

- `/auditar-claude-md`, `/dominio`, `/contrato`, `/planejar-setup`, `/spec-close`, `/session-start`, `/readme`, `/diagrama`, `/encerrar-projeto`. A maior parte é conferência mecânica que segue regras escritas. Um teste pequeno do `/spec-close` (6 de 6 cenários) foi descartado: era pequeno e montado para passar.
- O limite de 2 voltas de correção no `/implementar`. A pesquisa apoia um teto: cada volta a mais aumenta a trapaça de 33% para 38% (ImpossibleBench).
- O relatório "cumpre a spec?". A pesquisa diz para mostrar a evidência em vez de afirmar o sucesso (guia da Factory), mas ainda falta ver se ele muda alguma decisão na prática.